

5LSLO Machine Learning for signal processing - assignment 1

Gheorghiu Cristian Stefan*

Student number: 2036967

1 EXERCISE 1.3

1.1 1.3.a Requirement

Write a function that takes two strings as inputs, representing paths to stored weights. The function should return two object (i.e., your denoiser and your detector with the weights loaded from the files)

1.2 1.3.a Answer

Initially, one function taking two strings as inputs, representing paths to stored weights, was put in place. However, due to the very comprehensive nature of my class, this function was adequately augmented to support modes, each meant to put into action a certain setup (e.g., Fig 1 – training and obtaining a “better denoiser” based on a composite loss that factors in the detector; Fig 2 – applying the denoiser on the test set to create a list of denoised images, and consequently using the detector on the clean images, the noisy images, and the denoised images). **Only the most insightful parts of these class functions were attached since an extended view remains possible only by close inspection of the exercise₃.py**

```
1 def activate_Super_Training_Denoiser_Mode(self, epochs, alpha=0.1, detector_path=None, denoiser_path=None):
2     """
3     Train a denoiser against the frozen Super Detector.
4
5     Objective:
6     total_detector_loss = total_loss + reconstruction_loss + alpha * detector_loss
7     BCE loss of the frozen detector when denoised images are labelled as noisy.
8     Since we subtract this term, the denoiser is trained to increase detector loss,
9     therefore fooling the detector.
10
11     Only the denoiser parameters are updated.
12     """
13
14     for cl, ns, label in self.train_loader:
15         denoised_images = denoiser(ns)
16         reconstruction_loss = reconstruction_loss_fn(
17             denoised_images,
18             cl)
19
20         # 4.1 Detector loss
21
22         # Denoised noisy images originate from noisy images.
23         # so target is 0 from the detector's perspective.
24         # ... But we subtract this loss, forcing the denoiser!
25         # to make detector loss go up.
26
27         detector_output = detector(denoised_images)
28         noisy_targets = torch.zeros_like(detector_output)
29         detector_loss = detector_loss_fn(
30             detector_output,
31             noisy_targets)
32
33         # 5 Combined adversarial denoiser objective
34         total_loss = reconstruction_loss + alpha * detector_loss
```

Figure 1: Better denoiser training class function signature and composite loss reveal

1.3 1.3.b Requirement

Load the best version of your denoiser and the best version of your detector from the previous exercises. Apply the denoiser on the test set to create a list of denoised images. Then, apply your detector on the clean images, the noisy images, and the denoised images. Create a figure with three boxplots, showing the likelihood of each image being noisy in each set, as estimated by your noise detector.

1.4 1.3.b Answer

Fig. 3 – detector intuition on authentic clean and noisy images – depicts 3 separate boxplots showcasing how well (confidently) the model is distinguishing between classes belonging to any of these three datasets: clean and noisy images, plus denoised images.

*e-mail: c.gheorghiu.l@student.tue.nl

```
1 def activate_Super_Training_Detector_Mode(self, epochs):
2     # 1.1 Creates directories & initialize backbone architecture
3     self.utils.Directory.Exists()
4     detector = NoisyImageIdentifier.Net(
5         activation=[LeakyReLU, Sigmoid],
6         width=[32*2, 24*2, 22*2, 20*2, 22*2, 24*2, 32*2, 1]
7     )
8     # 1.2 Load the pre-trained denoiser:
9     denoiser = BasicConv(
10         activation=[LeakyReLU, Tanh],
11         width=[32*2, 24*2, 22*2, 20*2, 22*2, 24*2, 32*2]
12     )
13     print('Loading pretrained denoiser...')
14     denoiser_path = rf assignment_1_{2036967}_{2036967}Submission\exercise1
15     _3_first_half_results\train_denoiser_3_submission\train_denoiser\train_denoisermodel_weights_epoch
16     h10.pth
17     denoiser.load_state_dict(torch.load(denoiser_path))
18     denoiser.eval()
19
20     for epoch in range(epochs):
21         # 1.4.1 Training starts
22         detector.train()
23         _loss_per_epoch.tr(float = 0.0)
24         for cl, ns, label in self.train_loader:
25             current_b_size = cl.shape[0] # retrieving batch size
26             # 1.4.2 Feeding denoiser with ns - batch holding noisy images
27             with torch.no_grad():
28                 denoised_ns = denoiser(ns)
29
30             # 1.4.3 Random choice for noisy, vs denoised
31             # ... generates a one dimensional tensor holding randomly generated numbers (0,1)
32             # ... then by comparison < 0.5 those numbers will be converted to boolean flags (True / False)
33             # ... then use boolean indexing to create a mixture of noisy authentic images and
34             # reconstruction images
35             use_denoised_mask = torch.rand(current_b_size) < 0.5
36             augmented_noisy = ns.clone()
37             augmented_noisy[use_denoised_mask] = denoised_ns[use_denoised_mask]
```

Figure 2: Better detector training class function signature and augmentation mechanism

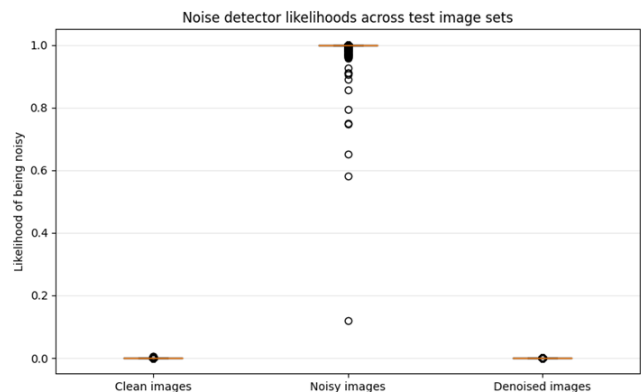


Figure 3: Detector output on authentic clean and noisy images

1.5 1.3.c Requirement

For each image, calculate whether your detector was “fooled” by your denoiser (i.e., whether the estimated likelihood changed). Make a boxplot of the changes

1.6 1.3.c Answer

As shown in Fig. 4, the change in detector likelihood after denoising is nothing less than a “drastic” change, making the proof of a model not sensitive enough to varying levels of noise. This is equivalent to saying that some of the denoised images, still holding less noise than before, were labeled inappropriately by the model as not noisy.

1.7 1.3.d Requirement

Use plt.subplots to create a 3x3 grid showing the samples where your detector was “the most fooled”, “the least fooled”, and a sample that corresponds to the median change in estimated likelihood. Show the clean, noisy and denoised images, and label each image with the estimated likelihoods

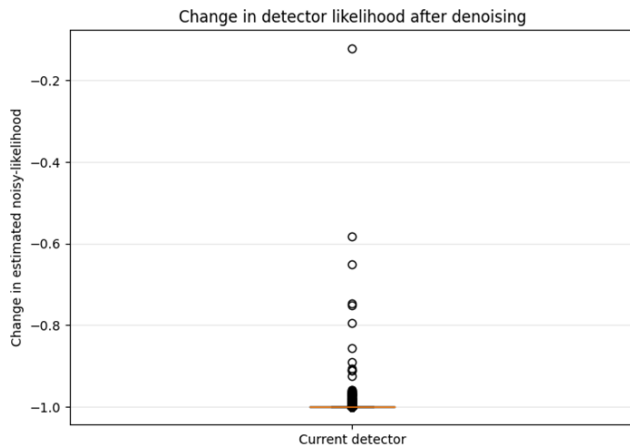


Figure 4: Change in the detector likelihood after denoising

1.8 1.3.d Answer

Fig. 5 – shows what the detector was actually labeling as 0 – not noisy – or 1 – noisy. As one might expect, the samples it deems not to be corrupted with noise are not necessarily entirely noise-free (e.g., the sample located at row 3, col 3 still holds quite a bit of noise, yet the detector confidently asserts it's not a noisy instance). These figures count as evidence for the previous premise: the previously trained detector is not accustomed to varying levels of noise (e.g., $\alpha = 0.1 \dots 0.9$).

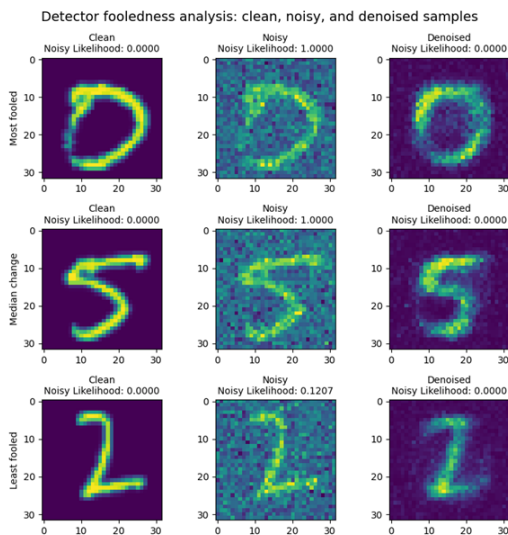


Figure 5: Samples representing most and least fooled, plus the median change

1.9 1.3.e Requirement

e) Now use the denoiser to make a better detector! Write a new training loop for your detector that uses your denoiser as an augmentation for the training samples. 50% of the time, use the original noisy training images and 50% of the time, run the noisy inputs through your denoiser first. Remember, that your optimizer should only optimize the weights in the detector, not the denoiser.

1.10 1.3.e Answer

In order to obtain a setup as described in the aforementioned requirement a different class method titled `activate_Super_Training_Detector_Mode()` was created, which as shown in Fig 2 is internally randomly augmenting the dataset and assembling (e.g., concatenating) clean images (cl) with noisy images:

Algorithm 1: Aug: Variable Assignment

```
augmented_noisy ← (cl, denoised);
return augmented_noisy;
```

1.11 1.3.f Requirement

After training, how do the final validation metric compare to the ones found in (2d)? Does that match your expectations? Explain why or why not in ≤ 3 sentences

1.12 1.3.f Answer

In contrast with the original detector, when equipping the super detector, a tremendous improvement becomes apparent that is clearly visible in Fig. 7—Change in detector likelihood after denoising. By seeing augmented samples, the detector became sensible to a new “in-between” distribution it has never learned to handle. In this case we get a higher accuracy score (almost infinitesimally 0.000154) that is indeed matching my expectations since the Super Detector is aware of a widened data distribution compared to before.



Figure 6: Super detector intuition on authentic clean and noisy images, plus denoised images

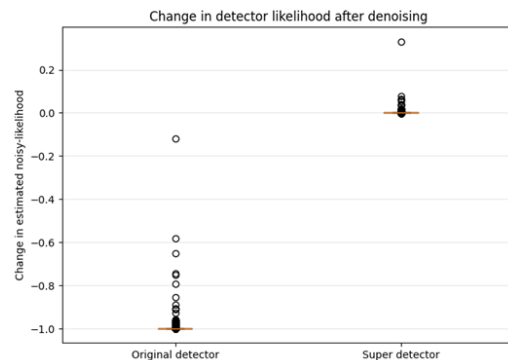


Figure 7: Change in detector likelihood after denoising with super-detector

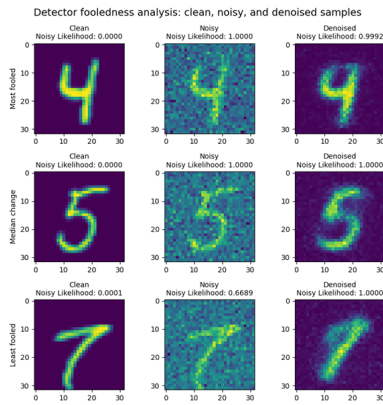


Figure 8: Samples representing most and least fooled, plus the median change with super-detector

1.13 1.3.h Requirement

Now use your new detector to make a better denoiser! Write a new training loop for your denoiser that runs the output through your detector. Compute a loss for the detector output, multiply it by α , and then add it to the loss of your denoiser. We want to train our denoiser to fool the detector, so the detection loss should go up, not down. Remember that your optimizer should only optimize the weights in the denoiser, not the detector. Plot both components of the loss as separate lines.

1.14 1.3.h Results



Figure 9: Default alpha level $\alpha = 0.1$

1.15 1.3.i Requirement

Depending on the loss functions you chose for both of your networks, different values for α may be appropriate. Explain how you chose your α (optionally support your answer with a figure of experimental results)

1.16 1.3.i Answer

Programming: As shown in Fig. 1—Class Method used to build a better denoiser—another class method taking as arguments the alpha level, detector_path and denoiser_path was put together for this purpose. As required, within the training loop, the composite loss has been defined as follows: $\text{total_loss} = \text{reconstruction_loss} - \alpha * \text{detector_loss}$

Experiment: In order to carry out this experiment, I adopted and tested for the following alpha values = [0.01, 0.05, 0.5, 0.8], for which four trials came about. With alpha = 0.1, the first plot in Fig. 10 shows that the detector loss suddenly explodes from near 0 to

roughly 90–95 around epoch 7. That means the denoiser found a way to make the detector very wrong. But that does not guarantee the image became visually better (e.g., as it happens in VAEs, the model collapse is replicable within the context as defined by this example as well). From the least high alpha level (0.05) Fig. 11 to the highest (0.8) 12, we observe that the detector's sudden loss increase takes place way earlier (the second epoch), which led to the following statement: **Higher alpha usually increases pressure to fool the detector, so detector loss may increase. However, if alpha is too large, the denoiser may sacrifice reconstruction quality to fool the detector. Therefore, better-looking reconstructions require a balanced alpha, not necessarily a larger alpha.**

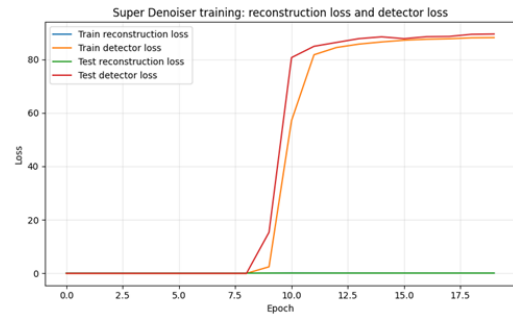


Figure 10: Lowest alpha level $\alpha = 0.05$



Figure 11: Medium alpha level $\alpha = 0.5$



Figure 12: Highest alpha level $\alpha = 0.8$

1.17 1.3.j Requirement

Generate figures like in (3g) with your new detector and your new denoiser. For the figure showing the change in estimated likelihoods, also include boxplots with the data from (3c) and (3g) for easier comparison.

1.18 1.3.j Answer

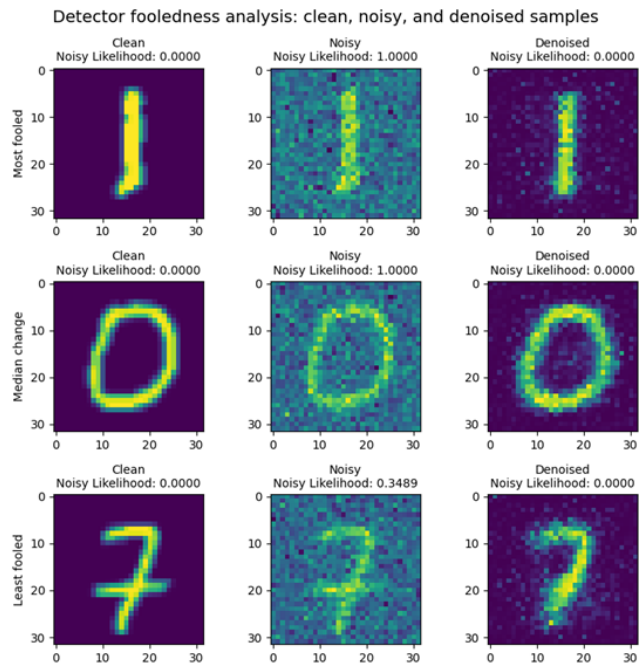


Figure 13: Visual Comparison

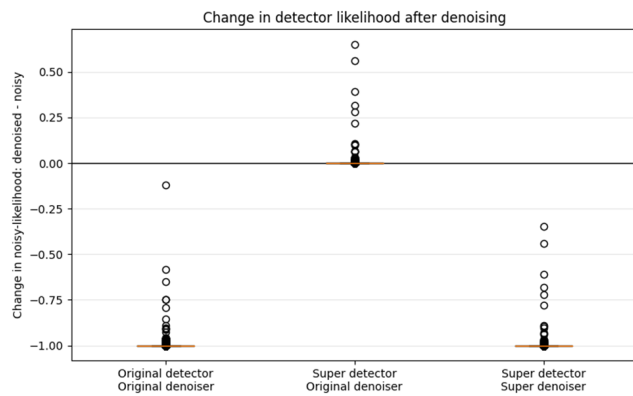


Figure 14: Pair wise likelihood comparison